

[< ALL GUIDES](#)

Building REST services with Spring

Get the Code

 [Go To Repo](#)

REST has quickly become the de facto standard for building web services on the web because REST services are easy to build and easy to consume.

A much larger discussion can be had about how REST fits in the world of microservices. However, for this tutorial, we look only at building RESTful services.

Why REST? REST embraces the precepts of the web, including its architecture, benefits, and everything else. This is no surprise, given that its author (Roy Fielding) was involved in probably a dozen specs which govern how the web operates.

What benefits? The web and its core protocol, HTTP, provide a stack of features:

- Suitable actions (`GET` , `POST` , `PUT` , `DELETE` , and others)
- Caching
- Redirection and forwarding
- Security (encryption and authentication)

These are all critical factors when building resilient services. However, that is not all. The web is built out of lots of tiny specs. This architecture lets it easily evolve without getting bogged down in "standards wars".

Developers can draw upon third-party toolkits that implement these different specs and instantly have both client and server technology at their fingertips.



- Backwards compatible APIs
- Evolvable APIs
- Scalable services
- Securable services
- A spectrum of stateless to stateful services

Note that REST, however ubiquitous, is not a standard *per se* but an approach, a style, a set of *constraints* on your architecture that can help you build web-scale systems. This tutorial uses the Spring portfolio to build a RESTful service while taking advantage of the stackless features of REST.

Getting Started

To get started, you need:

- A favorite text editor or IDE, such as:
 - [IntelliJ IDEA](#)
 - [VSCode](#)
- [Java 17](#) or later

As we work through this tutorial, we use [Spring Boot](#). Go to [Spring Initializr](#) and add the following dependencies to a project:

- Spring Web
- Spring Data JPA
- H2 Database

Change the Name to "Payroll" and then choose **Generate Project**. A [.zip](#) file downloads. Unzip it. Inside, you should find a simple, Maven-based project that includes a [pom.xml](#) build file. (Note: You can use Gradle. The examples in this tutorial will be Maven-based.)

To complete the tutorial, you could start a new project from scratch, or you could look at the [solution repository](#) in GitHub.



modules.

Rather than providing a single, final solution, the [completed GitHub repository](#) uses modules to separate the solution into four parts. The modules in the GitHub solution repository build on one another, with the `links` module containing the final solution. The modules map to the following headers:

- [The Story So Far \(nonrest\)](#)
- [Spring HATEOAS \(rest\)](#)
- [Simplifying Link Creation \(evolution\)](#)
- [Building links into your REST API \(links\)](#)

The Story so Far

This tutorial starts by building up the code in the `nonrest` module.

We start off with the simplest thing we can construct. In fact, to make it as simple as possible, we can even leave out the concepts of REST. (Later on, we add REST, to understand the difference.)

Big picture: We are going to create a simple payroll service that manages the employees of a company. We store employee objects in a (H2 in-memory) database, and access them (through something called JPA). Then we wrap that with something that allows access over the internet (called the Spring MVC layer).

The following code defines an `Employee` in our system.

▼ nonrest/src/main/java/payroll/Employee.java

```
package payroll;
```

[COPY](#)

```
import java.util.Objects;
```



```
import jakarta.persistence.Id;

@Entity
class Employee {

    private @Id
    @GeneratedValue Long id;
    private String name;
    private String role;

    Employee() {}

    Employee(String name, String role) {

        this.name = name;
        this.role = role;
    }

    public Long getId() {
        return this.id;
    }

    public String getName() {
        return this.name;
    }

    public String getRole() {
        return this.role;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public void setName(String name) {
        this.name = name;
    }
}
```



```
public void setRole(String role) {
    this.role = role;
}

@Override
public boolean equals(Object o) {

    if (this == o)
        return true;
    if (!(o instanceof Employee))
        return false;

    Employee employee = (Employee) o;
    return Objects.equals(this.id, employee.id) &&
Objects.equals(this.name, employee.name)
    && Objects.equals(this.role, employee.role);
}

@Override
public int hashCode() {
    return Objects.hash(this.id, this.name, this.role);
}

@Override
public String toString() {
    return "Employee{" + "id=" + this.id + ", name='" +
this.name + '\'' + ", role='" + this.role + '\'' + '}';
}
}
```

Despite being small, this Java class contains much:

- `@Entity` is a JPA annotation to make this object ready for storage in a JPA-based data store.
- `id`, `name`, and `role` are attributes of our `Employee` domain object. `id` is marked with more JPA annotations to indicate that it



- A custom constructor is created when we need to create a new instance but do not yet have an `id`.

With this domain object definition, we can now turn to [Spring Data JPA](#) to handle the tedious database interactions.

Spring Data JPA repositories are interfaces with methods that support creating, reading, updating, and deleting records against a back end data store. Some repositories also support data paging and sorting, where appropriate. Spring Data synthesizes implementations based on conventions found in the naming of the methods in the interface.

There are multiple repository implementations besides JPA. You can use [Spring Data MongoDB](#), [Spring Data Cassandra](#), and others. This tutorial sticks with JPA.

Spring makes accessing data easy. By declaring the following `EmployeeRepository` interface, we can automatically:

- Create new employees
- Update existing employees
- Delete employees
- Find employees (one, all, or search by simple or complex properties)

▼ nonrest/src/main/java/payroll/EmployeeRepository.java

```
package payroll;

import org.springframework.data.jpa.repository.JpaRepository;

interface EmployeeRepository extends JpaRepository<Employee,
Long> {
```

[COPY](#)



To get all this free functionality, all we have to do is declare an interface that extends Spring Data JPA's `JpaRepository`, specifying the domain type as `Employee` and the `id` type as `Long`.

Spring Data's [repository solution](#) makes it possible to sidestep data store specifics and, instead, solve a majority of problems by using domain-specific terminology.

Believe it or not, this is enough to launch an application! A Spring Boot application is, at a minimum, a `public static void main` entry-point and the `@SpringBootApplication` annotation. This tells Spring Boot to help out wherever possible.

▼ nonrest/src/main/java/payroll/PayrollApplication.java

```
package payroll;

import org.springframework.boot.SpringApplication;
import
org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class PayrollApplication {

    public static void main(String... args) {
        SpringApplication.run(PayrollApplication.class, args);
    }
}
```

[COPY](#)

`@SpringBootApplication` is a meta-annotation that pulls in **component scanning**, **auto-configuration**, and **property support**. We do not dive into the details of Spring Boot in this tutorial. However, in essence, it starts a servlet container and serves up our service.

An application with no data is not very interesting, so we preload that it has data. The following class gets loaded automatically by Spring:

▼ nonrest/src/main/java/payroll/LoadDatabase.java



```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.CommandLineRunner;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
class LoadDatabase {

    private static final Logger log =
        LoggerFactory.getLogger(LoadDatabase.class);

    @Bean
    CommandLineRunner initDatabase(EmployeeRepository repository)
    {

        return args -> {
            log.info("Preloading " + repository.save(new
Employee("Bilbo Baggins", "burglar")));
            log.info("Preloading " + repository.save(new
Employee("Frodo Baggins", "thief")));
        };
    }
}
```

What happens when it gets loaded?

- Spring Boot runs ALL `CommandLineRunner` beans once the application context is loaded.
- This runner requests a copy of the `EmployeeRepository` you just created.
- The runner creates two entities and stores them.

Right-click and **Run** `PayRollApplication`, and you get:



ment of console output showing preloading of data



```
20yy-08-09 11:36:26.105 INFO 74611 --- [main] payroll.LoadDatabase
20yy-08-09 11:36:26.174 INFO 74611 --- [main] payroll.LoadDatabase
...
```

This is not the **whole** log, but only the key bits of preloading data.

HTTP is the Platform

To wrap your repository with a web layer, you must turn to Spring MVC. Thanks to Spring Boot, you need add only a little code. Instead, we can focus on actions:

▼ nonrest/src/main/java/payroll/EmployeeController.java

```
package payroll;

import java.util.List;

import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RestController;

@RestController

class EmployeeController {

    private final EmployeeRepository repository;

    EmployeeController(EmployeeRepository repository) {
        this.repository = repository;
    }
}
```

[COPY](#)



```
@GetMapping("/employees")
List<Employee> all() {
    return repository.findAll();
}

// end::get-aggregate-root[]

@PostMapping("/employees")
Employee newEmployee(@RequestBody Employee newEmployee) {
    return repository.save(newEmployee);
}

// Single item

@GetMapping("/employees/{id}")
Employee one(@PathVariable Long id) {

    return repository.findById(id)
        .orElseThrow(() -> new EmployeeNotFoundException(id));
}

@PutMapping("/employees/{id}")
Employee replaceEmployee(@RequestBody Employee newEmployee,
    @PathVariable Long id) {

    return repository.findById(id)
        .map(employee -> {
            employee.setName(newEmployee.getName());
            employee.setRole(newEmployee.getRole());
            return repository.save(employee);
        })
        .orElseGet(() -> {
            return repository.save(newEmployee);
        });
}

deleteMapping("/employees/{id}")
id deleteEmployee(@PathVariable Long id) {
```



```
}
```

- `@RestController` indicates that the data returned by each method is written straight into the response body instead of rendering a template.
- An `EmployeeRepository` is injected by constructor into the controller.
- We have routes for each operation (`@GetMapping` , `@PostMapping` , `@PutMapping` and `@DeleteMapping` , corresponding to HTTP `GET` , `POST` , `PUT` , and `DELETE` calls). (We recommend reading each method and understanding what they do.)
- `EmployeeNotFoundException` is an exception used to indicate when an employee is looked up but not found.



nonrest/src/main/java/payroll/EmployeeNotFoundException.java

```
package payroll;

class EmployeeNotFoundException extends RuntimeException {

    EmployeeNotFoundException(Long id) {
        super("Could not find employee " + id);
    }
}
```

[COPY](#)

When an `EmployeeNotFoundException` is thrown, this extra tidbit of Spring MVC configuration is used to render an **HTTP 404** error:



nonrest/src/main/java/payroll/EmployeeNotFoundAdvice.java

```
package payroll;

import org.springframework.http.HttpStatus;
```

[COPY](#)



```
import org.springframework.web.bind.annotation.ResponseStatus;
import
org.springframework.web.bind.annotation.RestControllerAdvice;

@RestControllerAdvice

class EmployeeNotFoundAdvice {

    @ExceptionHandler(EmployeeNotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    String employeeNotFoundHandler(EmployeeNotFoundException ex)
    {
        return ex.getMessage();
    }
}
```

- `@RestControllerAdvice` signals that this advice is rendered straight into the response body.
- `@ExceptionHandler` configures the advice to only respond when an `EmployeeNotFoundException` is thrown.
- `@ResponseStatus` says to issue an `HttpStatus.NOT_FOUND` — that is, an **HTTP 404** error.
- The body of the advice generates the content. In this case, it gives the message of the exception.

To launch the application, you can right-click the

`public static void main` in `PayRollApplication` and select **Run** from your IDE.

Alternatively, Spring Initializr creates a Maven wrapper, so you can run the following command:

```
$ ./mvnw clean spring-boot:run
```

Alternatively, you can use your installed Maven version, as follows:





When the app starts, you can immediately interrogate it, as follows:

```
$ curl -v localhost:8080/employees
```

Doing so yields the following:

▼ Details

```
* Trying ::1...
* TCP_NODELAY set
* Connected to localhost (::1) port 8080 (#0)
> GET /employees HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/7.54.0
> Accept: */*
>
< HTTP/1.1 200
< Content-Type: application/json; charset=UTF-8
< Transfer-Encoding: chunked
< Date: Thu, 09 Aug 20yy 17:58:00 GMT
<
* Connection #0 to host localhost left intact
[{"id":1,"name":"Bilbo Baggins","role":"burglar"}, {"id":2,"name"
```

You can see the preloaded data in a compacted format.

Now try to query a user that doesn't exist, as follows:

```
$ curl -v localhost:8080/employees/99
```

When you do so, you get the following output:

▼ Details

```
* Trying ::1...
* TCP_NODELAY set
```



```
> Host: localhost:8080
> User-Agent: curl/7.54.0
> Accept: */*
>
< HTTP/1.1 404
< Content-Type: text/plain; charset=UTF-8
< Content-Length: 26
< Date: Thu, 09 Aug 20yy 18:00:56 GMT
<
* Connection #0 to host localhost left intact
Could not find employee 99
```

This message nicely shows an **HTTP 404** error with the custom message:

```
Could not find employee 99 .
```

It is not hard to show the currently coded interactions.

If you use Windows command prompt to issue cURL commands, the following command probably does not work properly. You must either pick a terminal that support single-quoted arguments, or use double quotation marks and then escape the quotation marks inside the JSON.

To create a new `Employee` record, use the following command in a terminal (the `$` at the beginning signifies that what follows it is a terminal command):

```
$ curl -X POST localhost:8080/employees -H 'Content-type:application/json' -d '{"id":3,"name":"Samwise Gamgee","role":"gardener"}'
```

Then it stores the newly created employee and sends it back to us:

```
{ "id": 3, "name": "Samwise Gamgee", "role": "gardener" }
```



```
$ curl -X PUT localhost:8080/employees/3 -H 'Content-type:application/json'
```

Now we can see the change reflected in the output:

```
{"id":3,"name":"Samwise Gamgee","role":"ring bearer"}
```

The way you construct your service can have significant impacts. In this situation, we said **update**, but **replace** is a better description. For example, if the name was NOT provided, it would instead get nulled out.

Finally, you can delete users, as follows:

```
$ curl -X DELETE localhost:8080/employees/3

# Now if we look again, it's gone
$ curl localhost:8080/employees/3
Could not find employee 3
```

This is all well and good, but do we have a RESTful service yet? (The answer is no.)

What's missing?

What Makes a Service RESTful?

So far, you have a web-based service that handles the core operations that involve employee data. However, that is not enough to make things "RESTful".

- Pretty URLs, such as `/employees/3`, aren't REST.



Merely using `GET`, `POST`, and so on is not REST.



In fact, what we have built so far is better described as **RPC (Remote Procedure Call)**, because there is no way to know how to interact with this service. If you published this today, you would also have to write a document or host a developer's portal somewhere with all the details.

This statement of Roy Fielding's may further lend a clue to the difference between **REST** and **RPC**:

I am getting frustrated by the number of people calling any HTTP-based interface a REST API. Today's example is the SocialSite REST API. That is RPC. It screams RPC. There is so much coupling on display that it should be given an X rating.

What needs to be done to make the REST architectural style clear on the notion that hypertext is a constraint? In other words, if the engine of application state (and hence the API) is not being driven by hypertext, then it cannot be RESTful and cannot be a REST API. Period. Is there some broken manual somewhere that needs to be fixed?

— Roy Fielding

<https://roy.gbiv.com/untangled/2008/rest-apis-must-be-hypertext-driven>

The side effect of not including hypermedia in our representations is that clients must hard-code URIs to navigate the API. This leads to the same brittle nature that predated the rise of e-commerce on the web. It signifies that our JSON output needs a little help.

Spring HATEOAS

Now we can introduce [Spring HATEOAS](#), a Spring project aimed at helping you write hypermedia-driven outputs. To upgrade your service to being RESTful, add the following to your build:

If you are following along in the [solution repository](#), the next section switches to the [rest module](#).



Adding Spring HATEOAS to

dependencies

section of

pom.xml



```
<groupId>org.springframework.boot</groupId>  
<artifactId>spring-boot-starter-hateoas</artifactId>  
</dependency>
```

This tiny library gives us the constructs that define a RESTful service and then render it in an acceptable format for client consumption.

A critical ingredient to any RESTful service is adding [links](#) to relevant operations. To make your controller more RESTful, add links like the following to the existing `one` method in `EmployeeController`:

▼ Getting a single item resource

```
@GetMapping("/employees/{id}")  
EntityModel<Employee> one(@PathVariable Long id) {  
  
    Employee employee = repository.findById(id) //  
        .orElseThrow(() -> new EmployeeNotFoundException(id));  
  
    return EntityModel.of(employee, //  
  
        linkTo(methodOn(EmployeeController.class).one(id)).withSelfRel(  
            ),  
  
        linkTo(methodOn(EmployeeController.class).all()).withRel("emplo  
        yees"));  
}
```

[COPY](#)

You also need to include new imports:

▼ Details

```
import org.springframework.hateoas.EntityModel;  
import static  
    org.springframework.hateoas.server.mvc.WebMvcLinkBuilder.*;
```

[COPY](#)



methods from `WebMvcLinkBuilder` to build these links. If you are using Spring WebFlux in your project, you must instead use `WebFluxLinkBuilder`.

This is very similar to what we had before, but a few things have changed:

- The return type of the method has changed from `Employee` to `EntityModel<Employee>`. `EntityModel<T>` is a generic container from Spring HATEOAS that includes not only the data but a collection of links.
- `linkTo(methodOn(EmployeeController.class).one(id)).withSelfRel()` asks that Spring HATEOAS build a link to the `one` method of `EmployeeController` and flag it as a `self` link.
- `linkTo(methodOn(EmployeeController.class).all()).withRel("employees")` asks Spring HATEOAS to build a link to the aggregate root, `all()`, and call it "employees".

What do we mean by "build a link?" One of Spring HATEOAS's core types is `Link`. It includes a **URI** and a **rel** (relation). Links are what empower the web. Before the World Wide Web, other document systems would render information or links, but it was the linking of documents WITH this kind of relationship metadata that stitched the web together.

Roy Fielding encourages building APIs with the same techniques that made the web successful, and links are one of them.

If you restart the application and query the employee record of *Bilbo*, you get a slightly different response than earlier:

Curling prettier

When your curl output gets more complex it can become hard to read.





```
curl -v localhost:8080/employees/1 | json_pp
```

▼ RESTful representation of a single employee

```
{
  "id": 1,
  "name": "Bilbo Baggins",
  "role": "burglar",
  "_links": {
    "self": {
      "href": "http://localhost:8080/employees/1"
    },
    "employees": {
      "href": "http://localhost:8080/employees"
    }
  }
}
```

[COPY](#)

This decompressed output shows not only the data elements you saw earlier (`id` , `name` , and `role`) but also a `_links` entry that contains two URIs. This entire document is formatted using [HAL](#).

HAL is a lightweight [mediatype](#) that allows encoding not only data but also hypermedia controls, alerting consumers to other parts of the API to which they can navigate. In this case, there is a "self" link (kind of like a [this](#) statement in code) along with a link back to the **aggregate root**.

To make the aggregate root also be more RESTful, you want to include top level links while also including any RESTful components within.

So we modify the following (located in the `nonrest` module of the completed code):

▼ Getting an aggregate root





```

List<Employee> all() {
    return repository.findAll();
}

```

We want the following (located in the `rest` module of the completed code):

▼ Getting an aggregate root resource

```

@GetMapping("/employees")
CollectionModel<EntityModel<Employee>> all() {

    List<EntityModel<Employee>> employees =
repository.findAll().stream()
    .map(employee -> EntityModel.of(employee,

linkTo(methodOn(EmployeeController.class).one(employee.getId()))
).withSelfRel(),

linkTo(methodOn(EmployeeController.class).all()).withRel("employees")))
    .collect(Collectors.toList());

    return CollectionModel.of(employees,
linkTo(methodOn(EmployeeController.class).all()).withSelfRel())
;
}

```

[COPY](#)

That method, which used to be merely `repository.findAll()`, is "all grown up." Not to worry. Now we can unpack it.

`CollectionModel<>` is another Spring HATEOAS container. It is aimed at encapsulating collections of resources instead of a single resource entity, such as `EntityModel<>` from earlier. `CollectionModel<>`, too, lets you include links.

Do not let that first statement slip by. What does "encapsulating collections" mean? Collections of employees?



Since we are talking REST, it should encapsulate collections or **employee resources**.

That is why you fetch all the employees but then transform them into a list of `EntityModel<Employee>` objects. (Thanks Java Streams!)

If you restart the application and fetch the aggregate root, you can see what it looks like now:

```
curl -v localhost:8080/employees | json_pp
```

▼ RESTful representation of a collection of employee resources

```
{
  "_embedded": {
    "employeeList": [
      {
        "id": 1,
        "name": "Bilbo Baggins",
        "role": "burglar",
        "_links": {
          "self": {
            "href": "http://localhost:8080/employees/1"
          },
          "employees": {
            "href": "http://localhost:8080/employees"
          }
        }
      },
      {
        "id": 2,
        "name": "Frodo Baggins",
        "role": "thief",
        "_links": {
          "self": {
            "href": "http://localhost:8080/employees/2"
          },
          "employees": {
```

[COPY](#)



```
    }  
  }  
]  
,  
"_links": {  
  "self": {  
    "href": "http://localhost:8080/employees"  
  }  
}
```

For this aggregate root, which serves up a collection of employee resources, there is a top-level **"self"** link. The **"collection"** is listed underneath the **"_embedded"** section. This is how HAL represents collections.

Each individual member of the collection has their information as well as related links.

What is the point of adding all these links? It makes it possible to evolve REST services over time. Existing links can be maintained while new links can be added in the future. Newer clients may take advantage of the new links, while legacy clients can sustain themselves on the old links. This is especially helpful if services get relocated and moved around. As long as the link structure is maintained, clients can still find and interact with things.

Simplifying Link Creation

If you are following along in the [solution repository](#), the next section switches to the [evolution module](#).

In the code earlier, did you notice the repetition in single employee link creation? The code to provide a single link to an employee, as well as to



You need to define a function that converts `Employee` objects to `EntityModel<Employee>` objects. While you could easily code this method yourself, Spring HATEOAS's `RepresentationModelAssembler` interface does the work for you. Create a new class `EmployeeModelAssembler`:

▼
evolution/src/main/java/payroll/EmployeeModelAssembler.java
a

```
package payroll;

import static
org.springframework.hateoas.server.mvc.WebMvcLinkBuilder.*;

import org.springframework.hateoas.EntityModel;
import
org.springframework.hateoas.server.RepresentationModelAssembler
;
import org.springframework.stereotype.Component;

@Component
class EmployeeModelAssembler implements
RepresentationModelAssembler<Employee, EntityModel<Employee>> {

    @Override
    public EntityModel<Employee> toModel(Employee employee) {

        return EntityModel.of(employee, //

linkTo(methodOn(EmployeeController.class).one(employee.getId()))
).withSelfRel(),

linkTo(methodOn(EmployeeController.class).all()).withRel("emplo
yees"));
}
```

[COPY](#)



(`EntityModel<Employee>`).

All the code you saw earlier in the controller can be moved into this class. Also, by applying Spring Framework's `@Component` annotation, the assembler is automatically created when the app starts.

Spring HATEOAS's abstract base class for all models is `RepresentationModel`. However, for simplicity, we recommend using `EntityModel<T>` as your mechanism to easily wrap all POJOs as models.

To leverage this assembler, you have only to alter the `EmployeeController` by injecting the assembler in the constructor:

▼ Injecting EmployeeModelAssembler into the controller

```
@RestController
class EmployeeController {

    private final EmployeeRepository repository;

    private final EmployeeModelAssembler assembler;

    EmployeeController(EmployeeRepository repository,
EmployeeModelAssembler assembler) {

        this.repository = repository;
        this.assembler = assembler;
    }

    ...

}
```

[COPY](#)



▼ Getting single item resource using the assembler

```
@GetMapping("/employees/{id}")
EntityModel<Employee> one(@PathVariable Long id) {

    Employee employee = repository.findById(id) //
        .orElseThrow(() -> new EmployeeNotFoundException(id));

    return assembler.toModel(employee);
}
```

This code is almost the same, except that, instead of creating the `EntityModel<Employee>` instance here, you delegate it to the assembler. Maybe that is not impressive.

Applying the same thing in the aggregate root controller method is more impressive. This change is also to the `EmployeeController` class:

▼ Getting aggregate root resource using the assembler

```
@GetMapping("/employees")
CollectionModel<EntityModel<Employee>> all() {

    List<EntityModel<Employee>> employees =
        repository.findAll().stream() //
            .map(assembler::toModel) //
            .collect(Collectors.toList());

    return CollectionModel.of(employees,
        linkTo(methodOn(EmployeeController.class).all()).withSelfRel());
}
```

[COPY](#)

The code is, again, almost the same. However, you get to replace all that `EntityModel<Employee>` creation logic with `map(assembler::toModel)`.



A key design goal of Spring HATEOAS is to make it easier to do The Right Thing™. In this scenario, that means adding hypermedia to your service without hard coding a thing.

At this stage, you have created a Spring MVC REST controller that actually produces hypermedia-powered content. Clients that do not speak HAL can ignore the extra bits while consuming the pure data. Clients that do speak HAL can navigate your empowered API.

But that is not the only thing needed to build a truly RESTful service with Spring.

Evolving REST APIs

With one additional library and a few lines of extra code, you have added hypermedia to your application. But that is not the only thing needed to make your service RESTful. An important facet of REST is the fact that it is neither a technology stack nor a single standard.

REST is a collection of architectural constraints that, when adopted, make your application much more resilient. A key factor of resilience is that when you make upgrades to your services, your clients do not suffer downtime.

In the "olden" days, upgrades were notorious for breaking clients. In other words, an upgrade to the server required an update to the client. In this day and age, hours or even minutes of downtime spent doing an upgrade can cost millions in lost revenue.

Some companies require that you present management with a plan to minimize downtime. In the past, you could get away with upgrading at 2:00 a.m. on a Sunday when load was at a minimum. But in today's Internet-based e-commerce with international customers in other time zones, such strategies are not as effective.



clients. With REST-based practices, it is much easier, especially using the Spring stack.

Supporting Changes to the API

Imagine this design problem: You have rolled out a system with this `Employee`-based record. The system is a major hit. You have sold your system to countless enterprises. Suddenly, the need for an employee's name to be split into `firstName` and `lastName` arises.

Uh oh. You did not think of that.

Before you open up the `Employee` class and replace the single field `name` with `firstName` and `lastName`, stop and think. Does that break any clients? How long will it take to upgrade them? Do you even control all the clients accessing your services?

Downtime = lost money. Is management ready for that?

There is an old strategy that precedes REST by years.

Never delete a column in a database.

— Unknown

You can always add columns (fields) to a database table. But do not take one away. The principle in RESTful services is the same.

Add new fields to your JSON representations, but do not take any away. Like this:

▼ JSON that supports multiple clients

```
{
  "id": 1,
  "firstName": "Bilbo",
  "lastName": "Baggins",
  "role": "burglar",
  "name": "Bilbo Baggins",
  "_links": {
    "self": {
      "href": "http://localhost:8080/employees/1"
    }
  }
}
```

[COPY](#)



```
}  
  
}  
  
}
```

This format shows `firstName`, `lastName`, and `name`. While it sports duplication of information, the purpose is to support both old and new clients. That means you can upgrade the server without requiring clients to upgrade at the same time. This is good move that should reduce downtime.

Not only should you show this information in both the "old way" and the "new way", but you should also process incoming data both ways.

▼ Employee record that handles both "old" and "new" clients

```
package payroll;  
  
import java.util.Objects;  
  
import jakarta.persistence.Entity;  
import jakarta.persistence.GeneratedValue;  
import jakarta.persistence.Id;  
  
@Entity  
class Employee {  
  
    private @Id @GeneratedValue Long id;  
    private String firstName;  
    private String lastName;  
    private String role;  
  
    Employee() {}  
  
    Employee(String firstName, String lastName, String role) {  
  
        this.firstName = firstName;  
        this.lastName = lastName;  
    }  
}
```

[COPY](#)



```
public String getName() {
    return this.firstName + " " + this.lastName;
}

public void setName(String name) {
    String[] parts = name.split(" ");
    this.firstName = parts[0];
    this.lastName = parts[1];
}

public Long getId() {
    return this.id;
}

public String getFirstName() {
    return this.firstName;
}

public String getLastName() {
    return this.lastName;
}

public String getRole() {
    return this.role;
}

public void setId(Long id) {
    this.id = id;
}

public void setFirstName(String firstName) {
    this.firstName = firstName;
}

public void setLastName(String lastName) {
```



```

public void setRole(String role) {
    this.role = role;
}

@Override
public boolean equals(Object o) {

    if (this == o)
        return true;
    if (!(o instanceof Employee))
        return false;
    Employee employee = (Employee) o;
    return Objects.equals(this.id, employee.id) &&
Objects.equals(this.firstName, employee.firstName)
        && Objects.equals(this.lastName, employee.lastName) &&
Objects.equals(this.role, employee.role);
}

@Override
public int hashCode() {
    return Objects.hash(this.id, this.firstName, this.lastName,
this.role);
}

@Override
public String toString() {
    return "Employee{" + "id=" + this.id + ", firstName='" +
this.firstName + '\'' + ", lastName='" + this.lastName
        + '\'' + ", role='" + this.role + '\'' + '}';
}
}

```

This class is similar to the previous version of `Employee`, with a few changes:



Field `name` has been replaced by `firstName` and `lastName`.



value.

- A "virtual" setter for the old `name` property, `setName()`, is also defined. It parses an incoming string and stores it into the proper fields.

Of course, not change to your API is as simple as splitting a string or merging two strings. But it is surely not impossible to come up with a set of transforms for most scenarios, right?

Do not forget to change how you preload your database (in `LoadDatabase`) to use this new constructor.

```
log.info("Preloading " + repository.save(new  
Employee("Bilbo", "Baggins", "burglar")));  
log.info("Preloading " + repository.save(new  
Employee("Frodo", "Baggins", "thief")));
```

[COPY](#)

Proper Responses

Another step in the right direction involves ensuring that each of your REST methods returns a proper response. Update the POST method (`newEmployee`) in the `EmployeeController`:

▼ POST that handles "old" and "new" client requests

```
@PostMapping("/employees")  
ResponseBody<?> newEmployee(@RequestBody Employee  
newEmployee) {  
  
    EntityModel<Employee> entityModel =  
    assembler.toModel(repository.save(newEmployee));  
  
    return ResponseEntity //
```

[COPY](#)



```
.body(entityModel);  
}
```

You also need to add the imports:

▼ Details

```
import org.springframework.hateoas.IanaLinkRelations;  
import org.springframework.http.ResponseEntity;
```

[COPY](#)

- The new `Employee` object is saved, as before. However, the resulting object is wrapped in the `EmployeeModelAssembler`.
- Spring MVC's `ResponseEntity` is used to create an **HTTP 201 Created** status message. This type of response typically includes a **Location** response header, and we use the URI derived from the model's self-related link.
- Additionally, the model-based version of the saved object is returned.

With these tweaks in place, you can use the same endpoint to create a new employee resource and use the legacy `name` field:

```
$ curl -v -X POST localhost:8080/employees -H 'Content-Type:appl:
```

The output is as follows:

▼ Details

```
> POST /employees HTTP/1.1  
> Host: localhost:8080  
> User-Agent: curl/7.54.0  
> Accept: */*  
> Content-Type:application/json  
> Content-Length: 46
```




```
< Transfer-Encoding: chunked
< Date: Fri, 10 Aug 20yy 19:44:43 GMT
<
{
  "id": 3,
  "firstName": "Samwise",
  "lastName": "Gamgee",
  "role": "gardener",
  "name": "Samwise Gamgee",
  "_links": {
    "self": {
      "href": "http://localhost:8080/employees/3"
    },
    "employees": {
      "href": "http://localhost:8080/employees"
    }
  }
}
```

This not only has the resulting object rendered in HAL (both `name` as well as `firstName` and `lastName`), but also the **Location** header populated with `http://localhost:8080/employees/3`. A hypermedia-powered client could opt to "surf" to this new resource and proceed to interact with it.

The PUT controller method (`replaceEmployee`) in `EmployeeController` needs similar tweaks:

▼ Handling a PUT for different clients

```
@PutMapping("/employees/{id}")
ResponseBody<> replaceEmployee(@RequestBody Employee
newEmployee, @PathVariable Long id) {

    Employee updatedEmployee = repository.findById(id) //
        .map(employee -> {
            employee.setName(newEmployee.getName());
```

[COPY](#)



```

    }) //
    .orElseGet(() -> {
        return repository.save(newEmployee);
    });

    EntityModel<Employee> entityModel =
    assembler.toModel(updatedEmployee);

    return ResponseEntity //

    .created(entityModel.getRequiredLink(IanaLinkRelations.SELF).to
    Uri()) //
    .body(entityModel);
}

```

The `Employee` object built by the `save()` operation is then wrapped in the `EmployeeModelAssembler` to create an `EntityModel<Employee>` object. Using the `getRequiredLink()` method, you can retrieve the `Link` created by the `EmployeeModelAssembler` with a `SELF` rel. This method returns a `Link`, which must be turned into a `URI` with the `toUri` method.

Since we want a more detailed HTTP response code than **200 OK**, we use Spring MVC's `ResponseEntity` wrapper. It has a handy static method (`created()`) where we can plug in the resource's URI. It is debatable whether **HTTP 201 Created** carries the right semantics, since we do not necessarily "create" a new resource. However, it comes preloaded with a **Location** response header, so we use it. Restart your application, run the following command, and observe the results:

```
$ curl -v -X PUT localhost:8080/employees/3 -H 'Content-Type:app
```

▼ Details

```

NODELAY set
connected to localhost (:::1) port 8080 (#0)

```



```
> User-Agent: curl/7.54.0
> Accept: */*
> Content-Type:application/json
> Content-Length: 49
>
< HTTP/1.1 201
< Location: http://localhost:8080/employees/3
< Content-Type: application/hal+json;charset=UTF-8
< Transfer-Encoding: chunked
< Date: Fri, 10 Aug 20yy 19:52:56 GMT
{
    "id": 3,
    "firstName": "Samwise",
    "lastName": "Gamgee",
    "role": "ring bearer",
    "name": "Samwise Gamgee",
    "_links": {
        "self": {
            "href": "http://localhost:8080/employees/3"
        },
        "employees": {
            "href": "http://localhost:8080/employees/"
        }
    }
}
```

That employee resource has now been updated and the location URI has

been set to `http://localhost:8080/employees/3`.

in `EmployeeController`:

▼ Handling DELETE requests

```
@DeleteMapping("/employees/{id}")
ResponseBody deleteEmployee(@PathVariable Long id) {
    repository.deleteById(id);
}
```

[COPY](#)



```
}
```

This returns an **HTTP 204 No Content** response. Restart your application, run the following command, and observe the results:

```
$ curl -v -X DELETE localhost:8080/employees/1
```

▼ Details

```
* TCP_NODELAY set
* Connected to localhost (:::1) port 8080 (#0)
> DELETE /employees/1 HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/7.54.0
> Accept: */*
>
< HTTP/1.1 204
< Date: Fri, 10 Aug 20yy 21:30:26 GMT
```

Making changes to the fields in the `Employee` class requires coordination with your database team, so that they can properly migrate existing content into the new columns.

You are now ready for an upgrade that does not disturb existing clients while newer clients can take advantage of the enhancements.

By the way, are you worried about sending too much information over the wire? In some systems where every byte counts, evolution of APIs may need to take a backseat. However, you should not pursue such premature optimization until you measure the impact of your changes.

Building links into your REST API



section switches to the [links module](#).

So far, you have built an evolvable API with bare bones links. To grow your API and better serve your clients, you need to embrace the concept of **Hypermedia as the Engine of Application State**.

What does that mean? This section explores it in detail.

Business logic inevitably builds up rules that involve processes. The risk of such systems is we often carry such server-side logic into clients and build up strong coupling. REST is about breaking down such connections and minimizing such coupling.

To show how to cope with state changes without triggering breaking changes in clients, imagine adding a system that fulfills orders.

As a first step, define a new `Order` record:

▼ `links/src/main/java/payroll/Order.java`

```
package payroll;

import java.util.Objects;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "CUSTOMER_ORDER")
class Order {

    private @Id @GeneratedValue Long id;

    private String description;
    private Status status;
```

[COPY](#)



```
Order(String description, Status status) {

    this.description = description;
    this.status = status;
}

public Long getId() {
    return this.id;
}

public String getDescription() {
    return this.description;
}

public Status getStatus() {
    return this.status;
}

public void setId(Long id) {
    this.id = id;
}

public void setDescription(String description) {
    this.description = description;
}

public void setStatus(Status status) {
    this.status = status;
}

@Override
public boolean equals(Object o) {

    if (this == o)
        return true;
    if (!(o instanceof Order))
        return false;
```



```
Objects.equals(this.description, order.description)
    && this.status == order.status;
}

@Override
public int hashCode() {
    return Objects.hash(this.id, this.description,
this.status);
}

@Override
public String toString() {
    return "Order{" + "id=" + this.id + ", description='" +
this.description + '\'' + ", status=" + this.status + '}';
}
}
```

- The class requires a JPA `@Table` annotation that changes the table's name to `CUSTOMER_ORDER` because `ORDER` is not a valid name for table.
- It includes a `description` field as well as a `status` field.

Orders must go through a certain series of state transitions from the time a customer submits an order and it is either fulfilled or cancelled.

This can be captured as a Java `enum` called `Status`:

▼ `links/src/main/java/payroll/Status.java`

```
package payroll;
```

[COPY](#)

```
enum Status {
```

```
    IN_PROGRESS, //
```

```
    COMPLETED, //
```

```
    CANCELLED
```



To support interacting with orders in the database, you must define a corresponding Spring Data repository called `OrderRepository`:

▼ Spring Data JPA's `JpaRepository` base interface

```
interface OrderRepository extends JpaRepository<Order, Long> {  
}
```

[COPY](#)

We also need to create a new exception class called

`OrderNotFoundException`:

▼ Details

```
package payroll;  
  
class OrderNotFoundException extends RuntimeException {  
  
    OrderNotFoundException(Long id) {  
        super("Could not find order " + id);  
    }  
}
```

[COPY](#)

With this in place, you can now define a basic `OrderController` with the required imports:

▼ Import Statements

```
import java.util.List;  
import java.util.stream.Collectors;  
  
import static  
    org.springframework.hateoas.server.mvc.WebMvcLinkBuilder.*;  
  
import org.springframework.hateoas.CollectionModel;  
import org.springframework.hateoas.EntityModel;  
import org.springframework.http.ResponseEntity;
```

[COPY](#)



```
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RestController;
```

▼ links/src/main/java/payroll/OrderController.java

```
@RestController
class OrderController {

    private final OrderRepository orderRepository;
    private final OrderModelAssembler assembler;

    OrderController(OrderRepository orderRepository,
OrderModelAssembler assembler) {

        this.orderRepository = orderRepository;
        this.assembler = assembler;
    }

    @GetMapping("/orders")
    CollectionModel<EntityModel<Order>> all() {

        List<EntityModel<Order>> orders =
orderRepository.findAll().stream() //
        .map(assembler::toModel) //
        .collect(Collectors.toList());

        return CollectionModel.of(orders, //

linkTo(methodOn(OrderController.class).all()).withSelfRel());
    }

    @GetMapping("/orders/{id}")
    EntityModel<Order> one(@PathVariable Long id) {

        Order order = orderRepository.findById(id) //
```

[COPY](#)



```

    return assembler.toModel(order);
}

@PostMapping("/orders")
ResponseBody<EntityModel<Order>> newOrder(@RequestBody
Order order) {

    order.setStatus(Status.IN_PROGRESS);
    Order newOrder = orderRepository.save(order);

    return ResponseEntity //

        .created(linkTo(methodOn(OrderController.class).one(newOrder.getId()))
        .toUri()) //
        .body(assembler.toModel(newOrder));
}
}

```

- It contains the same REST controller setup as the controllers you have built so far.
- It injects both an `OrderRepository` and a (not yet built) `OrderModelAssembler`.
- The first two Spring MVC routes handle the aggregate root as well as a single item `Order` resource request.
- The third Spring MVC route handles creating new orders, by starting them in the `IN_PROGRESS` state.
- All the controller methods return one of Spring HATEOAS's `RepresentationModel` subclasses to properly render hypermedia (or a wrapper around such a type).

Before building the `OrderModelAssembler`, we should discuss what needs to happen. You are modeling the flow of states between `Status.IN_PROGRESS`, `Status.COMPLETED`, and `Status.CANCELLED`. A natural thing when serving up such data to clients is to let the clients make the decision about what they can do, based on this payload.



What happens when you introduce a new state in this flow? The placement of various buttons on the UI would probably be erroneous.

What if you changed the name of each state, perhaps while coding international support and showing locale-specific text for each state? That would most likely break all the clients.

Enter **HATEOAS** or **Hypermedia as the Engine of Application State**.

Instead of clients parsing the payload, give them links to signal valid actions. Decouple state-based actions from the payload of data. In other words, when **CANCEL** and **COMPLETE** are valid actions, you should dynamically add them to the list of links. Clients need to show users the corresponding buttons only when the links exist.

This decouples clients from having to know when such actions are valid, reducing the risk of the server and its clients getting out of sync on the logic of state transitions.

Having already embraced the concept of Spring HATEOAS

`RepresentationModelAssembler` components, the

`OrderModelAssembler` is the perfect place to capture the logic for this business rule:

▼ `links/src/main/java/payroll/OrderModelAssembler.java`

```
package payroll;

import static
    org.springframework.hateoas.server.mvc.WebMvcLinkBuilder.*;

import org.springframework.hateoas.EntityModel;
import
    org.springframework.hateoas.server.RepresentationModelAssembler
;
import org.springframework.stereotype.Component;

@Component
class OrderModelAssembler implements
    RepresentationModelAssembler<Order, EntityModel<Order>> {
```

[COPY](#)



```
// Unconditional links to single-item resource and
aggregate root

EntityModel<Order> orderModel = EntityModel.of(order,

linkTo(methodOn(OrderController.class).one(order.getId())).with
SelfRel(),

linkTo(methodOn(OrderController.class).all()).withRel("orders")
);

// Conditional links based on state of the order

if (order.getStatus() == Status.IN_PROGRESS) {

orderModel.add(linkTo(methodOn(OrderController.class).cancel(or
der.getId())).withRel("cancel"));

orderModel.add(linkTo(methodOn(OrderController.class).complete(
order.getId())).withRel("complete"));

}

return orderModel;
}
}
```

This resource assembler always includes the **self** link to the single-item resource as well as a link back to the aggregate root. However, it also includes two conditional links to `OrderController.cancel(id)` as well as `OrderController.complete(id)` (not yet defined). These links are shown only when the order's status is `Status.IN_PROGRESS`.

If clients can adopt HAL and the ability to read links instead of simply reading the data of plain old JSON, they can trade in the need for domain knowledge about the order system. This naturally reduces



To round out order fulfillment, add the following to the `OrderController` for the `cancel` operation:

▼ Creating a "cancel" operation in the OrderController

```
@DeleteMapping("/orders/{id}/cancel")
ResponseBody<?> cancel(@PathVariable Long id) {

    Order order = orderRepository.findById(id) //
        .orElseThrow(() -> new OrderNotFoundException(id));

    if (order.getStatus() == Status.IN_PROGRESS) {
        order.setStatus(Status.CANCELLED);
        return
    }

    ResponseEntity.ok(assembly.toModel(orderRepository.save(order)
    ));

    return ResponseEntity //
        .status(HttpStatus.METHOD_NOT_ALLOWED) //
        .header(HttpHeaders.CONTENT_TYPE,
        MediaType.APPLICATION_PROBLEM_JSON) //
        .body(Problem.create() //
            .withTitle("Method not allowed") //
            .withDetail("You can't cancel an order that is in the
            " + order.getStatus() + " status"));
}
```

COPY

It checks the `Order` status before letting it be cancelled. If it is not a valid state, it returns an [RFC-7807 Problem](#), a hypermedia-supporting error container. If the transition is indeed valid, it transitions the `Order` to `CANCELLED`.

Now we need to add this to the `OrderController` as well for order completion:

▼ Creating a "complete" operation in the OrderController



```

responseEntity = complete(@PathVariable Long id) {

    Order order = orderRepository.findById(id) //
        .orElseThrow(() -> new OrderNotFoundException(id));

    if (order.getStatus() == Status.IN_PROGRESS) {
        order.setStatus(Status.COMPLETED);
        return
    }

    ResponseEntity.ok(assembly.toModel(orderRepository.save(order)
    ));

}

return ResponseEntity //
    .status(HttpStatus.METHOD_NOT_ALLOWED) //
    .header(HttpHeaders.CONTENT_TYPE,
    MediaType.TEXT_PROBLEM_DETAILS_JSON_VALUE) //
    .body(Problem.create() //
        .withTitle("Method not allowed") //
        .withDetail("You can't complete an order that is in
the " + order.getStatus() + " status"));
}

```

This implements similar logic to prevent an `Order` status from being completed unless in the proper state.

Let's update `LoadDatabase` to preload some `Order` objects along with the `Employee` objects it was loading before.

▼ Updating the database preloader

```

package payroll;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.CommandLineRunner;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

```

[COPY](#)



```
private static final Logger log =
LoggerFactory.getLogger(LoadDatabase.class);

@Bean
CommandLineRunner initDatabase(EmployeeRepository
employeeRepository, OrderRepository orderRepository) {

    return args -> {
        employeeRepository.save(new Employee("Bilbo", "Baggins",
"burglar"));
        employeeRepository.save(new Employee("Frodo", "Baggins",
"thief"));

        employeeRepository.findAll().forEach(employee ->
log.info("Preloaded " + employee));

        orderRepository.save(new Order("MacBook Pro",
Status.COMPLETED));
        orderRepository.save(new Order("iPhone",
Status.IN_PROGRESS));

        orderRepository.findAll().forEach(order -> {
            log.info("Preloaded " + order);
        });
    };
}
```

Now you can test. Restart your application to make sure you are running the latest code changes. To use the newly minted order service, you can perform a few operations:

```
curl -v http://localhost:8080/orders | json_pp
```

```
{
  "_embedded": {
    "orderList": [
      {
        "id": 3,
        "description": "MacBook Pro",
        "status": "COMPLETED",
        "_links": {
          "self": {
            "href": "http://localhost:8080/orders/3"
          },
          "orders": {
            "href": "http://localhost:8080/orders"
          }
        }
      },
      {
        "id": 4,
        "description": "iPhone",
        "status": "IN_PROGRESS",
        "_links": {
          "self": {
            "href": "http://localhost:8080/orders/4"
          },
          "orders": {
            "href": "http://localhost:8080/orders"
          },
          "cancel": {
            "href": "http://localhost:8080/orders/4/cancel"
          },
          "complete": {
            "href": "http://localhost:8080/orders/4/complete"
          }
        }
      }
    ]
  }
}
```




```
"self": {  
  "href": "http://localhost:8080/orders"  
}  
}  
}
```

This HAL document immediately shows different links for each order, based upon its present state.

- The first order, being **COMPLETED**, only has the navigational links. The state transition links are not shown.
- The second order, being **IN_PROGRESS**, additionally has the **cancel** link as well as the **complete** link.

Now try cancelling an order:

```
$ curl -v -X DELETE http://localhost:8080/orders/4/cancel | json_
```

You may need to replace the number 4 in the preceding URL, based on the specific IDs in your database. That information can be found from the previous `/orders` call.

▼ Details

```
> DELETE /orders/4/cancel HTTP/1.1  
> Host: localhost:8080  
> User-Agent: curl/7.54.0  
> Accept: */*  
>  
< HTTP/1.1 200  
< Content-Type: application/hal+json;charset=UTF-8  
< Transfer-Encoding: chunked  
< Date: Mon, 27 Aug 20yy 15:02:10 GMT
```



```
"id": 4,
"description": "iPhone",
"status": "CANCELLED",
"_links": {
  "self": {
    "href": "http://localhost:8080/orders/4"
  },
  "orders": {
    "href": "http://localhost:8080/orders"
  }
}
```

This response shows an **HTTP 200** status code, indicating that it was successful. The response HAL document shows that order in its new state ([CANCELLED](#)). Also, the state-altering links gone.

Now try the same operation again:

```
$ curl -v -X DELETE http://localhost:8080/orders/4/cancel | json_
```

▼ Details

```
* TCP_NODELAY set
* Connected to localhost (:::1) port 8080 (#0)
> DELETE /orders/4/cancel HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/7.54.0
> Accept: */*
>
< HTTP/1.1 405
< Content-Type: application/problem+json
< Transfer-Encoding: chunked
< Date: Mon, 27 Aug 20yy 15:03:24 GMT
```



```
}
```

You can see an **HTTP 405 Method Not Allowed** response. **DELETE** has become an invalid operation. The `Problem` response object clearly indicates that you are not allowed to "cancel" an order already in the "CANCELLED" status.

Additionally, trying to complete the same order also fails:

```
$ curl -v -X PUT localhost:8080/orders/4/complete | json_pp
```

▼ Details

```
* TCP_NODELAY set
* Connected to localhost (:::1) port 8080 (#0)
> PUT /orders/4/complete HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/7.54.0
> Accept: */*
>
< HTTP/1.1 405
< Content-Type: application/problem+json
< Transfer-Encoding: chunked
< Date: Mon, 27 Aug 20yy 15:05:40 GMT
<
{
  "title": "Method not allowed",
  "detail": "You can't complete an order that is in the CANCELLED"
}
```

With all this in place, your order fulfillment service is capable of conditionally showing what operations are available. It also guards against invalid operations.

By using the protocol of hypermedia and links, clients can be made smarter and be less likely to break simply because of a change in the

Summary

Throughout this tutorial, you have engaged in various tactics to build REST APIs. As it turns out, REST is not just about pretty URIs and returning JSON instead of XML.

Instead, the following tactics help make your services less likely to break existing clients you may or may not control:

- Do not remove old fields. Instead, support them.
- Use rel-based links so clients need not hard code URIs.
- Retain old links as long as possible. Even if you have to change the URI, keep the rels so that older clients have a path to the newer features.
- Use links, not payload data, to instruct clients when various state-driving operations are available.

It may appear to be a bit of effort to build up

`RepresentationModelAssembler` implementations for each resource type and to use these components in all of your controllers. However, this extra bit of server-side setup (made easy thanks to Spring HATEOAS) can ensure the clients you control (and more importantly, those you do not control) can upgrade with ease as you evolve your API.

This concludes our tutorial on how to build RESTful services using Spring. Each section of this tutorial is managed as a separate subproject in a single github repo:

- **nonrest** — Simple Spring MVC app with no hypermedia
- **rest** — Spring MVC + Spring HATEOAS app with HAL representations of each resource
- **evolution** — REST app where a field is evolved but old data is retained for backward compatibility
- **links** — REST app where conditional links are used to signal valid state changes to clients



To do some more exploring, check out the following video by Spring teammate Oliver Drotbohm:

Configuração do player de vídeo

Erro 153

Erro 153

Want to write a new guide or contribute to an existing one? Check out our [contribution guidelines](#).

All guides are released with an ASLv2 license for the code, and an [Attribution, NoDerivatives creative commons license](#) for the writing.

Why Spring

Generative AI
Microservices
Reactive
Event Driven
Cloud
Web
Applications
Serverless
Batch

Learn

Quickstart
Guides
Courses
Get Certified

Projects

Resources

Blog
Release
Calendar
Version
Mappings
Release
Highlights
Security
Advisories

Community

Events
Authors
Enterprise
Long-term
Support
Automated
Upgrades
Governance and
Compliance
Modern App
Development

Thank You





Get the Spring newsletter

Stay connected with the Spring newsletter

SUBSCRIBE



Copyright © 2005 - 2026 Broadcom. All Rights Reserved. The term "Broadcom" refers to Broadcom Inc. and/or its subsidiaries.

[Terms of Use](#) • [Privacy](#) • [Trademark Guidelines](#)

Apache®, Apache Tomcat®, Apache Kafka®, Apache Cassandra™, and Apache Geode™ are trademarks or registered trademarks of the Apache Software Foundation in the United States and/or other countries. Java™, Java™ SE, Java™ EE, and OpenJDK™ are trademarks of Oracle and/or its affiliates. Kubernetes® is a registered trademark of the Linux Foundation in the United States and other countries. Linux® is the registered trademark of Linus Torvalds in the United States and other countries. Windows® and Microsoft® Azure are registered trademarks of Microsoft Corporation. "AWS" and "Amazon Web Services" are trademarks or registered trademarks of Amazon.com Inc. or its affiliates. All other trademarks and copyrights are property of their respective owners and are only mentioned for informative purposes. Other names may be trademarks of their respective owners.

